

Building an HR Recruitment Assistant

Using LangChain · Ollama · FAISS · RAG

AI-Powered HR Recruitment System

Overview

Business Scenario

A software company receives hundreds of resumes every month. The HR department wants an AI Assistant that can:

- Answer candidate questions
- Calculate years of experience
- Check candidate eligibility
- Generate interview questions
- Search company policy documents
- Provide intelligent responses using company knowledge

Your task is to design and implement this assistant using LangChain.

Learning Objectives

By completing this assignment, students will be able to:

- Work with Large Language Models (LLMs)
- Create Prompt Templates
- Build Custom Tools
- Develop Tool Calling Agents
- Use Memory for conversation history
- Generate Structured Outputs using Pydantic
- Implement RAG using FAISS and Embeddings
- Integrate multiple AI components into a single application

Problem Statement

A company HR team receives hundreds of resumes every day.

Employees and candidates frequently ask questions such as:

- What is the notice period?
- How many casual leaves are allowed?
- Am I eligible for the Python Developer role?
- What skills are required for a particular position?

The HR team spends significant time answering repetitive questions and manually screening candidates. The goal of this project is to build an AI assistant that automates these tasks.

Topics Covered

1. Large Language Models (LLMs)

Topics

- What is an LLM?
- Chat Models
- Local Models
- Ollama

Practical

- Connect to Qwen using Ollama
- Generate responses

2. Prompt Templates

Topics

- Dynamic prompts
- Reusable prompts
- Prompt engineering

Practical

- Build HR prompts
- Create reusable templates

3. Chains and LCEL

Topics

- What is a Chain?
- LCEL (LangChain Expression Language)

- Pipe Operator

Practical

- Prompt → LLM workflow

4. Tools

Topics

- Tool creation
- Tool execution
- Function calling

Practical

- Experience Calculator
- Eligibility Checker
- Interview Question Generator

5. Agents

Topics

- Agent Architecture
- Tool Selection
- Decision Making

Practical

- Build an HR Agent

6. Structured Output

Topics

- Pydantic Models
- JSON Outputs
- Data Validation

Practical

- Resume Parsing

7. Document Loaders

Topics

- Reading external files
- TextLoader
- PDF Loader

Practical

- Load company policy documents

8. Text Splitters

Topics

- Chunking
- Chunk Size
- Chunk Overlap

Practical

- Split large documents

9. Embeddings

Topics

- Semantic Search
- Vector Representations
- Embedding Models

Practical

- Create embeddings using `nomic-embed-text`

10. Vector Databases

Topics

- FAISS
- Similarity Search

Practical

- Store embeddings

11. Retrievers

Topics

- Context Retrieval
- Similarity Search

Practical

- Retrieve relevant chunks

12. RAG (Retrieval-Augmented Generation)

Topics

- Retrieval Augmented Generation
- Context Injection

Practical

- Answer questions from company documents

Functional Requirements

Phase 1 — HR Chatbot

Create a chatbot that answers candidate questions such as:

- What skills are required for a Python Developer?
- What are the responsibilities of a Software Engineer?

Use the following LangChain components:

- ChatOllama
- ChatPromptTemplate
- LCEL Chains

Phase 2 — Experience Calculator Tool

Create a LangChain Tool that calculates total years of experience from a candidate's start year.

Input	2020
Output	6 Years Experience

Phase 3 — Eligibility Checker Tool

Create a Tool that verifies whether a candidate possesses all required skills.

Required Skills: Python, SQL, Git

Input	Python, SQL, Git
Output	Eligible

Input	Python, SQL
Output	Not Eligible

Phase 4 — Tool Calling Agent

Create an Agent capable of automatically selecting the correct tool based on user queries.

Example Queries:

- I started working in 2020. How many years of experience do I have?
- Check whether I am eligible with Python, SQL and Git skills.

The Agent must decide which tool to invoke.

Phase 5 — Structured Output

Create a Pydantic model named Candidate with the following required fields:

- Name
- Experience
- Skills
- Eligibility Status

The LLM should return responses in structured format.

Phase 6 — Interview Question Generator

Create a module that generates interview questions based on candidate skills.

Input	Python, SQL
Output	Five relevant interview questions

Phase 7 — Memory Integration

Implement conversation memory so that the assistant remembers previous interactions.

User says	My name is Arun.
Later asks	What is my name?
Expected	Arun

Phase 8 — RAG-Based Policy Search

Create a knowledge base using company documents:

- LeavePolicy.txt / PDF
- EmployeeHandbook.txt / PDF
- PythonDeveloperJD.txt / PDF

Implement the following RAG pipeline components:

- Document Loader
- Text Splitter
- Embeddings
- FAISS Vector Database
- Retriever

The assistant should answer questions using company documents, such as:

- What is the notice period?
- How many casual leaves are allowed?
- What skills are required for a Python Developer?

Technical Requirements

Models

- qwen2.5:3b
- nomic-embed-text

LangChain Components (Mandatory)

- Prompt Templates
- Tools
- Agents
- Memory
- Structured Output
- Retriever
- RAG

Vector Database

- FAISS

Project Structure

```
HR_Recruitment_Assistant/  
├── hr_agent.py  
├── create_vector_db.py  
└── documents/  
    ├── LeavePolicy.txt  
    ├── EmployeeHandbook.txt  
    └── PythonDeveloperJD.txt
```

Documents Used

File	Contents
LeavePolicy.txt	Casual Leave Policy, Sick Leave Policy, Earned Leave Policy
EmployeeHandbook.txt	Working Hours, Notice Period, Work From Home Policy, Dress Code
PythonDeveloperJD.txt	Required Skills, Preferred Skills, Responsibilities, Experience Requirements

Test Cases

#	Input	Expected Output	Feature
1	I started working in 2021.	5 years of experience	Experience Calculator
2	Check eligibility for Python, SQL, Git.	Eligible	Eligibility Checker
3	Check eligibility for Python and SQL.	Missing Git skill	Eligibility Checker
4	What is the notice period?	60 days	RAG Policy Search
5	How many casual leaves are allowed?	12 casual leaves	RAG Policy Search
6	Generate interview questions for Python.	Five Python interview questions	Interview Generator
7	Resume: Name: Arun, Experience: 4, Skills: Python, SQL, Git	Structured Candidate Object	Structured Output

Software Requirements

Python Packages

```
pip install langchain
pip install langchain-ollama
pip install langchain-community
pip install langchain-core
pip install langchain-classic
pip install faiss-cpu
pip install pydantic
```

Ollama Models

```
ollama pull qwen2.5:3b
ollama pull nomic-embed-text
```

Source Code

hr_agent.py

```
from datetime import datetime
from typing import List
from pydantic import BaseModel
from langchain.tools import tool
from langchain_ollama import ChatOllama
from langchain_ollama import OllamaEmbeddings
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_community.vectorstores import FAISS
from langchain_classic.agents import (
    create_tool_calling_agent,
    AgentExecutor
)

# =====
# MODEL
# =====
llm = ChatOllama(model="qwen2.5:3b")

# =====
# MEMORY
# =====
chat_history = InMemoryChatMessageHistory()

# =====
# LOAD VECTOR DB
# =====
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vector_db = FAISS.load_local(
    "hr_vector_db",
    embeddings,
    allow_dangerous_deserialization=True
)
```

```

retriever = vector_db.as_retriever(search_kwargs={"k": 3})

# =====
# TOOLS
# =====
@tool
def experience_calculator(start_year: int) -> str:
    """Calculate candidate experience"""
    return str(datetime.now().year - start_year)

@tool
def eligibility_checker(skills: str) -> str:
    """Check candidate eligibility"""
    required = {"python", "sql", "git"}
    candidate = {skill.strip().lower() for skill in skills.split(",")}
    missing = required - candidate
    if len(missing) == 0:
        return "Eligible"
    return "Not Eligible. Missing: " + ", ".join(missing)

@tool
def company_policy_search(question: str) -> str:
    """Search company documents"""
    docs = retriever.invoke(question)
    context = "\n".join(doc.page_content for doc in docs)
    prompt = f"Answer only from\ncontext.\n\nContext:\n{context}\n\nQuestion:\n{question}"
    return llm.invoke(prompt).content

@tool
def interview_questions(skills: str) -> str:
    """Generate interview questions"""
    prompt = f"Generate 5 interview questions for: {skills}"
    return llm.invoke(prompt).content

tools = [experience_calculator, eligibility_checker,
         company_policy_search, interview_questions]

# =====
# PROMPT
# =====
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an HR Recruitment Assistant. Use tools whenever required."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

# =====
# AGENT
# =====
agent = create_tool_calling_agent(llm=llm, tools=tools, prompt=prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# =====
# STRUCTURED OUTPUT
# =====
class Candidate(BaseModel):
    name: str
    experience: int
    skills: List[str]

structured_llm = llm.with_structured_output(Candidate)

```

```
# =====
# CHAT LOOP
# =====
print("=" * 60)
print("HR RECRUITMENT ASSISTANT")
print("=" * 60)

while True:
    user_input = input("\nYou : ")
    if user_input.lower() == "exit":
        break
    if user_input.startswith("resume:"):
        resume = user_input.replace("resume:", "")
        candidate = structured_llm.invoke(f"Extract Name, Experience, Skills\n\nResume:\n{resume}")
        print("\nCandidate Details")
        print(candidate)
        continue
    response = agent_executor.invoke({"input": user_input})
    print("\nAssistant:", response["output"])
```

create_vector_db.py

```
import os
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_ollama import OllamaEmbeddings
from langchain_community.vectorstores import FAISS

# Load documents
documents = []
for file in os.listdir("documents"):
    path = os.path.join("documents", file)
    loader = TextLoader(path)
    docs = loader.load()
    documents.extend(docs)

print(f"Loaded {len(documents)} documents")

# Split documents
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(documents)
print(f"Created {len(chunks)} chunks")

# Create embeddings & vector DB
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vector_db = FAISS.from_documents(chunks, embeddings)
vector_db.save_local("hr_vector_db")
print("Vector Database Created")
```

Project Limitations

Current version limitations:

- No persistent memory

- No authentication
- No user management
- No workflow orchestration
- Single-agent architecture

Introduction to LangGraph

LangGraph is a framework used to build stateful AI workflows.

LangChain builds:

- Prompts
- Tools
- Agents
- RAG Systems

LangGraph builds:

- Multi-step workflows
- Human approval systems
- Multi-agent systems
- Conditional routing
- Stateful applications

Example Recruitment Workflow (LangGraph)

```
Resume Upload
  ↓
Resume Parsing
  ↓
Eligibility Check
  ↓
Eligible?
├── Yes → Generate Interview Questions
└── No  → Reject Candidate
```

Learning Outcomes

By completing this project, students will be able to:

- Understand LangChain architecture
- Work with local LLMs using Ollama
- Create Prompt Templates
- Build Chains using LCEL
- Create and use Tools
- Build AI Agents
- Generate Structured Outputs
- Load external documents
- Create Embeddings
- Build Vector Databases
- Implement Retrievers
- Build Retrieval-Augmented Generation (RAG) systems
- Understand LangGraph workflows

Expected Outcome

Students will complete an end-to-end AI application covering:

- LangChain Fundamentals
- Tools & Agents
- Structured Output
- Embeddings & Vector Databases
- Retrievers & RAG
- Introduction to LangGraph

This project mirrors the architecture used in modern enterprise AI assistants and serves as a strong foundation for advanced Agentic AI development.